

# 같은 목표, 두 가지 과정

## 카페 랜딩 한 페이지

한 문장 지시 (Vibe) vs 4 필드 스펙 (Spec), 같은 목표

화면은 거의 같고, 청구서와 검사 결과가 다르다

무엇이 비용을 움직였는지 하나씩 본다

# 사례 · 카페 랜딩 한 페이지

토큰 +1.8% · 비용 8.3% 감소

한 문장 지시 vs 4 필드 스펙 — 같은 목표

GPT-5 8.3% · Sonnet 7.6% · 캐시 미적용 8.1%

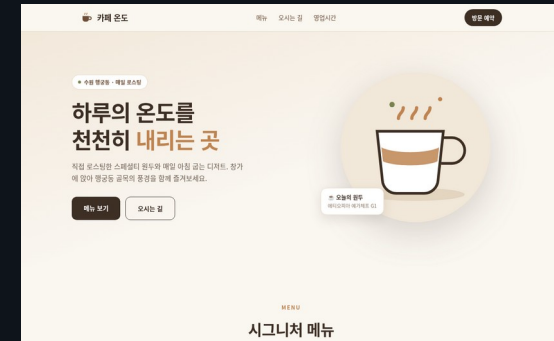
입력은 늘고, 비싼 출력은 줄었다

# 두 산출물의 첫 화면

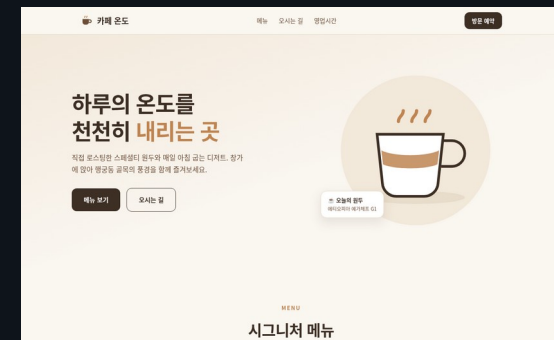
## 같은 목표, 다른 생성 과정

두 장 모두 실제 생성 HTML 을 Chromium 1280px 에서 캡처

다른 점은 토큰 구조와 검사 결과에 있다



위 : Vibe 산출물 첫 화면 (1280px)



아래 : Spec 산출물 첫 화면 (1280px)

demo/vibe\_vs\_spec/shots/vibe\_top.png · demo/vibe\_vs\_spec/shots/spec\_top.png

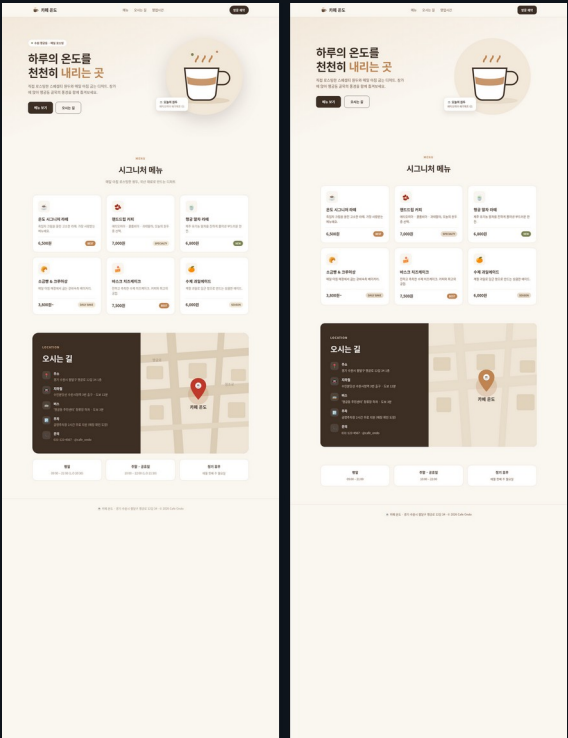
2026-09-20 · 근거와 조건을 함께 인용

# 전체 페이지 길이 비교

## 세로로 긴 한 페이지, 두 산출물

원본 캡처 두 장을 같은 높이로 축소해 나란히 붙였다

구성 요소 수와 배치가 비슷한지 눈으로 확인한다



왼쪽 Vibe 전체 페이지 · 오른쪽 Spec 전체 페이지 ( 축소 )

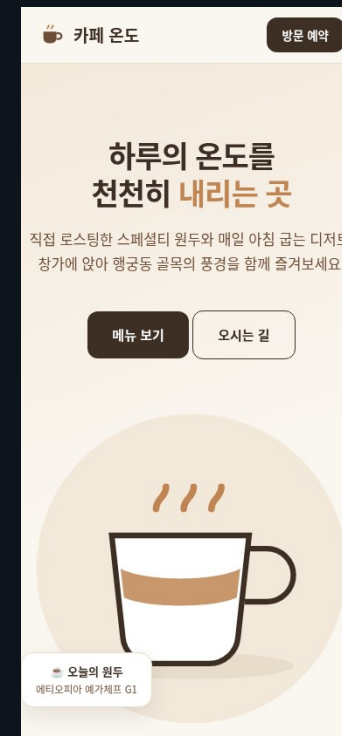
demo/vibe\_vs\_spec/shots/vibe\_full.png · demo/vibe\_vs\_spec/shots/spec\_full.png

# 모바일 390px 에서도 같은 1 열

## 정적 문자열이 아니라 실제 렌더링

문자열 검사만으로는 2 열로 바뀌어도 통과한다

아래 그림은 현재 Spec 산출물을 390px 에서 캡처한 것



Spec 산출물 · 390×844 캡처

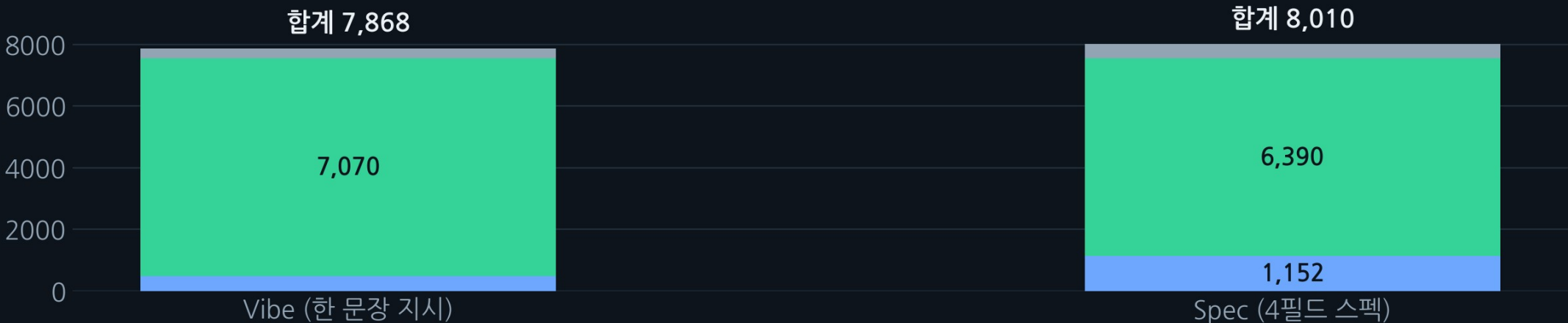
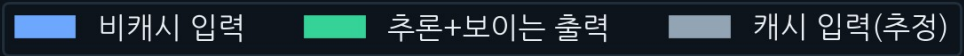
# 무엇이 비용을 움직였나

합계 7,868 → 8,010 (+1.8%)

비캐시 입력 ↑ · 추론 + 출력 ↓

합계는 거의 그대로 , 구조만 바뀌었다

두 산출물의 토큰 구조 (추정 usage)



# 입력이 늘고 비싼 출력이 줄었다

증가 666 · 감소 680 · 증가 156

이 케이스에서 input 과 cache 는 서로 겹치지 않는 버킷으로 정의

합계는 7,868 → 8,010 (증가 1.8%)

|             |                         |
|-------------|-------------------------|
| 비캐시 입력      | 486 → 1,152 (증가 666)    |
| 추론 + 보이는 출력 | 7,070 → 6,390 (감소 680)  |
| 캐시 입력 (추정)  | 312 → 468 (증가 156)      |
| 합계          | 7,868 → 8,010 (증가 1.8%) |

# 세 단가로 같은 추정치를 환산

8.3% / 7.6% / 8.1% 비용 감소

같은 추정 토큰을 세 단가로 환산한 표

모델을 바꿔도 방향은 같다

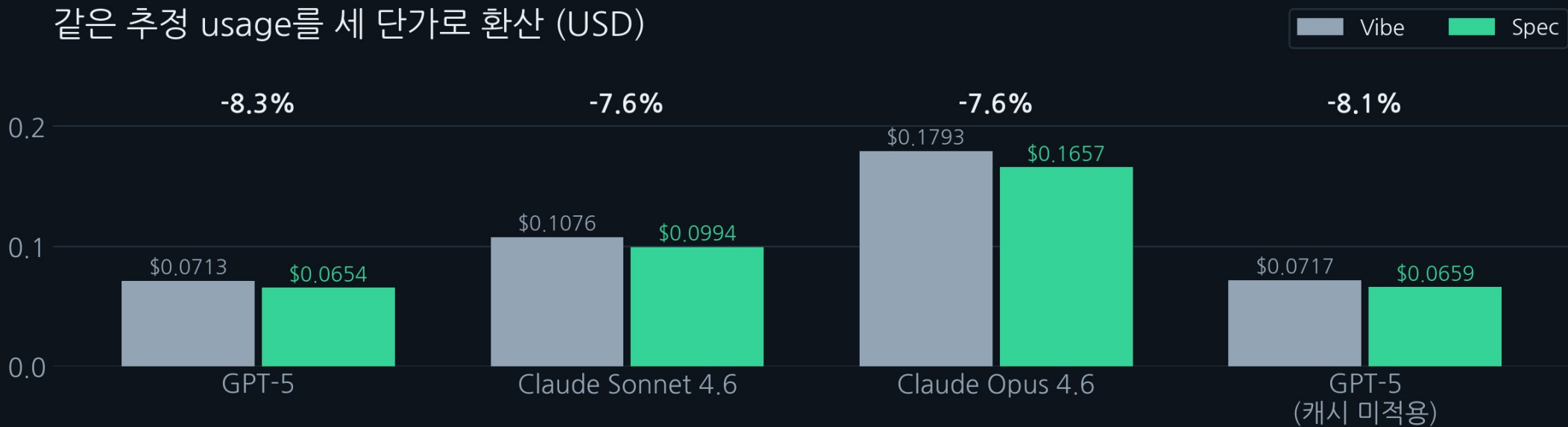
|                             |                                     |
|-----------------------------|-------------------------------------|
| GPT-5 · 캐시 \$0.125/M        | \$0.0713465 → \$0.0653985 (8.3% 감소) |
| Claude Sonnet 4.6 · 캐시 0.1× | \$0.1076016 → \$0.0994464 (7.6% 감소) |
| GPT-5 · 캐시 전부 미적용           | \$0.0716975 → \$0.0659250 (8.1% 감소) |

# 단가가 바뀌어도 방향은 같다

감소 8.3% · 7.6% · 8.1%

같은 추정 usage 를 세 단가로 환산

출력 단가가 높을수록 격차가 커진다



# 단가비가 손익을 결정한다

$r > (666 + 156h) / 680$

$r$  = 출력 / 입력 단가 ,  $h$  = 캐시 읽기 / 입력 단가

출력 단가가 조금만 높아도 스펙 쪽이 유리해진다

|                           |                          |
|---------------------------|--------------------------|
| 손익분기 조건                   | $r > (666 + 156h) / 680$ |
| $h = 0.1$ 이면              | $r = 1.0024$             |
| $r \rightarrow \infty$ 수렴 | 약 9.62% 감소에 수렴           |
| 실제 청구서                    | 미확인 (usage 추정치 산술 )      |

# 무엇을 확인했나

## 정적 16 + 브라우저 29 항목

390 · 820 · 821 · 1280px 렌더링과 앵커 이동 검사

실패하는 변조를 넣어 검사가 실제로 잡는지도 확인

# 그 네 칸에 무엇을 적는가

## 한 줄이면 충분한 결정 네 개

페이지 — 섹션과 순서를 고정한다 ( 이번엔 히어로 · 메뉴 · 지도 · 리뷰 · CTA )

스택 — 생성 방식과 파일 구성을 정한다 ( 단일 HTML + CSS, JS 없음 )

디자인 — 톤 · 색 · 타이포 기준을 한 줄로 ( 따뜻한 크림 톤 , 사진 대신 일러스트 )

수용 기준 — 검사 가능한 문장으로 ( 390px 에서 1 열 , 앵커 이동 , JS 오류 0 )

# 다음 프로젝트에서 할 일

## 결정 네 개를 먼저 적는다

페이지 · 스택 · 디자인 · 수용 기준

설명 대신 결정을 적고, 수용 기준을 코드로 옮긴다

완료 후 재작업 횟수와 총비용을 기록한다

# 재현

`python demo/vibe_vs_spec/verify_ac.py`

이미지 · usage 추정치 · 검사 스크립트가 저장소에 있다

브라우저가 없으면 UNVERIFIED 로 종료한다

덱과 대본도 같은 원천에서 생성된다

`demo/vibe_vs_spec/`

2026-09-20 · 근거와 조건을 함께 인용

# 마무리

## 스펙은 짧게 , 결정만

서론 대신 페이지 · 스택 · 디자인 · 수용 기준

되돌림이 줄면 턴 수가 줄고 비용도 줄어든다

다음 사례는 다수 과제 A/B 로 검증한다